

GOT: Generalization Over Taxonomies, a Software Toolkit for Content Analysis with Taxonomies

Dmitry Frolov¹ and Boris Mirkin^{1,2}

¹ Department of Data Analysis and Artificial Intelligence, HSE University, Moscow, Russia

² Department of Computer Science and Information Systems, Birkbeck University of London, London, UK
`{dfrolov,bmirkin}@hse.ru`

Abstract. GOT is a Python3 software toolkit for taxonomic content analysis of text collections. The structure of the toolkit follows an in-house methodology for processing a collection of texts using a domain taxonomy. The method includes the following steps: (1) computing matrix of relevance between texts and taxonomy leaf topics using a **purely** structural string-to-text relevance measure based on suffix trees representing the texts and annotated by substring frequencies, (2) obtaining fuzzy clusters of taxonomy leaf topics using a method involving both additive and spectral properties, and (3) finding most **specific** generalizations of the fuzzy clusters in a rooted tree of the taxonomy. Such a generalization **parsimoniously** lifts a cluster to its “head subject” in the higher ranks of the taxonomy, to **tightly** cover the cluster by minimizing the number of errors, “**gaps**” and “**offshoots**”. The efficiency of this methodology was illustrated in the analysis of research tendencies in Data Science: our findings led to insights on the tendencies of research that could not be derived by using more conventional techniques. The toolkit can be used either as a whole or with its individual modules including a visualization module. GOT toolkit provides for two usage scenarios: (a) console mode for using via command line and (b) import mode for using in Python3 source codes.

Keywords: Python3 toolkit, content analysis, hybrid approach, generalization, fuzzy cluster, annotated suffix tree

1 Introduction

Automation of text analysis is one of the major challenges of artificial intelligence. Despite serious successes associated with solving individual problems of text analysis, such as categorization, summarization, classification, etc., analysis of text collections’ contents in general remains a serious issue for researchers.

The closest approach to this, in our opinion, is the so-called “content analysis” [9]. It is a general method for identifying patterns in texts, based on selection

of certain categories and analysis of their occurrences. This approach was developed in the pre-digital era, primarily as a method of sociological research. However, it is currently used to analyze collections of texts in any field. Currently, this method has been greatly enriched by the use of multivariate data analysis methods including the so-called topic modeling [1,2]. The topic modeling model is applied to text documents represented as “bags of words” and is based on the extraction of so-called “hidden” topics, which are related in a probabilistic way to the observed word frequencies. As a result of applying the method, lists of words corresponding to separate topics are obtained, the meaning of which is restored “manually” from the analysis of the general meaning of the words included in them.

A typical topic modeling study is described in [3]. For the scientific field “ecological sociology”, the authors formed a set of 815 scientific publications and analyzed the contents of annotations to these publications using the method of thematic modeling. They determined, using the so-called perplexity index, that the best number of hidden topics for a given population would be 25 to 31, of which they chose 25 because the resulting topics were best interpreted. Examples of topics received in [3]: Environmental concern (environment, concern, attitude, result), Values (social, value, research, concept, relation), Policy & planning (public, management, group, decision, plan). Please note that the names of these topics were proposed by the human authors.

Themes formed by thematic modeling, in contrast to classical content analysis, are determined empirically, not theoretically, which presents certain problems in the way of theoretical understanding of the results. The approach of taxonomic content analysis, proposed by our team in [7,12], includes a theoretical component explicitly. This approach involves a pre-developed taxonomy of the subject area. Taxonomy is a tree-like hierarchy of basic concepts in which the fact that A is the “parent” of B in a taxonomy means the relation “ B is a part or a particular species of A ”. First, taxonomy supplies a list of prime categories, i.e. concepts of the lowest level of the hierarchy, the distribution structure of which relative to the collection of texts under consideration is the subject of analysis. Second, taxonomy exposes its top-level concepts as general topics inherent in a particular group (cluster) of prime categories.

2 Taxonomic content analysis

The method of taxonomic content analysis described in [7,12] works as follows. First of all (as always in content analysis) a collection of text documents is formed, representing a particular area of knowledge or activity. Then – or independently – a taxonomy of the subject area (domain) is formed, leaf topics of which represent the categories for a follow-up analysis.

After that, the following three stages are carried out sequentially:

- Category-to-text relevance matrix computation.
- Obtaining fuzzy clusters of categories according to the structure of matrices of relevance and co-relevance.

- Optimally lifting each of the clusters obtained.

More precisely, the first stage examines the representation of the lower-rank categories of taxonomy in the texts of the collection. The proposed measure of relevance between the text and the key phrase of a category is based on the alignment of their fragments, using the machinery of the so-called Annotated Suffix Trees (AST). AST calculates the average characteristic of the conditional probability of correct prediction of letters based on knowledge of the fragments used in overlays. This characteristic of the average conditional probability is used as a measure of relevance between the text and the category. This method, developed in [13,4,5,6], has shown its effectiveness in a number of experiments. This method works especially well in those fairly common situations when not all categories of the taxonomy concept system are present in texts under consideration. The computation of relevance matrices between texts, on the one hand, and categories, on the other hand, completes the first stage of our method.

The second stage is finding fuzzy clusters of categories. To do this, the matrix of relevance between texts and categories is transformed in a certain way into a matrix of co-relevance values between categories. After that, an additive-spectral FADDIS method [11] is applied. To sharpen the cluster structure of the matrix each text is weighted by a reciprocal number of the relevant categories. In addition, the Laplacian normalization from spectral cluster analysis [14,10] applies. The FADDIS method extracts clusters sequentially, one by one, so that it is always possible to stop the process of obtaining clusters and determine the number of clusters in this way. This is facilitated by characteristics of the contribution of clusters to the data scatter.

The third stage is finding of general meanings of the resulting clusters. Unlike other approaches, where a general meaning assigning to a cluster is a matter of imagination and ingenuity of the authors, the method proposed in [7,12] do this automatically using a special algorithm called ParGenFS. ParGenFS finds an optimal set of "head subjects" for a given fuzzy cluster of leaf nodes in taxonomy. The set of "head subjects" (one or more) contains a high-rank taxonomy concepts that includes (i.e. "covers") nodes from a given cluster, perhaps, with errors of two types: a small number of "gaps" and "offshoots" (see more detail in [7]). "Gap" is a node of the taxonomy tree that is covered by the head subject, but does not belong to the cluster under consideration, and "offshoot" is a node from the cluster that is not covered by the head subject. In many cases, this optimal lifting translates the structure under consideration into some significantly "coarser" level of granulation, which allows one to see some ideas and connections that are not obvious with more intuitive approaches.

The efficiency of this hybrid methodology was demonstrated in several publications. In [7,12], the methodology was applied to extract research tendencies from two collections of research papers: (a) about 18000 research papers published in Springer journals on data science for 20 years, and (b) about 27000 research papers retrieved from Springer and Elsevier journals in response to data science related queries. The authors used a taxonomy of Data Science (TDS) based on the Association for Computing Machinery Classification of Computing

System (ACM-CCS 2012). In [8], the generalization algorithm was applied for expanding advertising audience on the Internet.

3 Description and structure of GOT toolkit

A toolkit implementing the taxonomic content analysis methodology described above, the GOT (**G**eneralization **O**ver **T**axonomies) can be downloaded from GitHub repository³. The package provides for all steps of the method and lets the user to:

- Calculate string - to - text document relevance values, compose relevance and co-relevance matrices.
- Perform fuzzy clustering using our FADDIS algorithm implementation.
- Make optimal generalization of fuzzy clusters in taxonomy using our Par-GenFS method implementation.
- Visualize taxonomies and fuzzy cluster generalization results.

The package structure includes three modules containing program files:

- (a) *asts*,
- (b) *relevance_analysis*, and
- (c) *taxonomies*.

In the following subsections we will describe the structure of the GOT toolkit.

3.1 asts

This module includes a software implementation of the annotated suffix tree (AST), which allows to calculate string -to- text document relevance. The **ast.py** file contains the AST implementation. To calculate the relevance between a string and a text document using it, one should:

1. Create AST for a text using the *AST()* class constructor.
2. Call the *score()* method with an argument, the string for which you want to calculate relevance.

3.2 relevance_analysis

This module contains implementations of programs for obtaining relevance and relevance matrices, LAPIN transformations, and fuzzy clustering FADDIS.

The **relevance.py** file allows to get a relevance matrix of a set of strings of a collection of texts based on the annotated suffix tree method. This file contains functions for calculating the relevance matrix: *get_relevance_matrix* function allows to get a relevance matrix between a list of text documents and a list of strings.

³ <https://github.com/dmitsf/GOT/>

The **corelevance.py** file includes functions for calculating the relevance matrix between a list of text documents and a list of strings. The *get_corelevance_matrix* function allows to get a co-relevance matrix, the **lapin.py** file contains a function that performs the Laplace pseudo-inverse transform (LAPIN), and the **faddis.py** program contains a function for finding fuzzy clusters using the FADDIS method.

3.3 taxonomies

This module includes programs for working with taxonomies, generalizing fuzzy thematic clusters using the ParGenFS algorithm and visualizing the results. The **taxonomy.py** file brings together functions for working with taxonomy. Specifically, it combines parsing a taxonomy from a *.fvtr* file and preparing basic data structures to work with it. **fvtr** – (flat-view taxonomy representation) is a csv-like format for taxonomies’ representing. The format is based on the representation used in the ACM CCS taxonomy from the ACM official website⁴. Let’s take an example, a fragment of Internet Advertising Bureau (IAB) content taxonomy⁵ in *.fvtr* format. This taxonomy represents a hierarchy of content topics in the Internet.

```
579.,Men’s Fashion,
579.580.,Men’s Accessories,
579.580.581.,,,Men’s Jewelry and Watches,
579.582.,Men’s Clothing Style,
579.582.583.,,,Men’s Business Wear,
579.582.584.,,,Men’s Casual Wear,
579.582.585.,,,Men’s Formal Wear,
579.582.586.,,,Men’s Outerwear Style,
579.582.587.,,,Men’s Sportswear,
579.582.588.,,,Men’s Underwear and Sleepwear,
579.589.,Men’s Shoes and Footwear,
```

This fragment represents the taxonomy tree shown in Figure 1. You can find more examples of *.fvtr* files in the *test_files/* directory of the GOT repository.

To work with taxonomies, two classes are implemented: **Tree** represents a taxonomy tree, **Node** represents a node of a taxonomy tree. See the technical documentation⁶.

The **pargenfs.py** file contains functions that implement the algorithm for optimal generalization of a fuzzy cluster over a taxonomy (ParGenFS). After running, it creates two files, a table with the values of all variables at all stages of the algorithm, and a file with a lifting tree in *.ete* format (will be described below). Some examples of clusters’ generalizations over IAB taxonomy are presented in Table 1. These clusters contain users’ interests in the Internet, so we

⁴ <https://www.acm.org/publications/class-2012>

⁵ <https://www.iab.com/guidelines/content-taxonomy/>

⁶ <https://got-docs.readthedocs.io/en/latest/got.taxonomies.html>

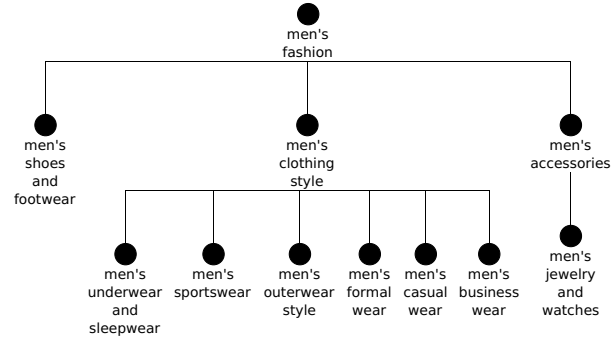


Fig. 1. IAB taxonomy fragment.

can obtain high-level representations of users' interests. This property was utilized in a special user audience extending algorithm, see the paper by Frolov, Taran and Mirkin [8], as was mentioned above.

.ete is a format for tree representation used in the ETE toolkit⁷ and other similar packages. The format allows to define the topology of the tree along with the attributes of the tree nodes. This format assumes bracket notation that makes it difficult to read by humans, but, on the other hand, it is extremely compact and easy to parse by programs. You can see an example of *.ete* tree representation from the ETE toolkit documentation below.

```

"(A,B,(C,D));"
#
#      /-A
#      |
#-----|--B
#      |
#      |      /-C
#      \----|
#            \-D

```

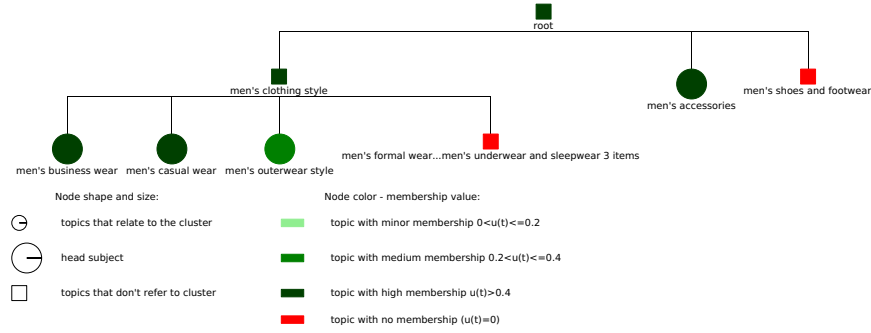
The **visualize.py** file contains functions for visualization of lifting trees in *.ete*. The module can produce images in *.pdf* format. An example of visualization of a cluster lifting over IAB taxonomy fragment mentioned above is shown in Figure 2. The module has a wide range of settings. It also allows to visualize

⁷ <http://etetoolkit.org/>

Table 1. Examples of fuzzy clusters’ generalizations over IAB taxonomy.

Fuzzy cluster with membership values	Generalization after ParGenFS applying
{Cloud Computing (0.596), Web Development (0.481), Internet for Beginners (0.432), IT and Internet Support (0.356), Social Networking (0.312)}	{Internet (1.0)}
{Men’s Jewelry and Watches (0.662), Men’s Business Wear (0.514), Men’s Casual Wear (0.443), Men’s Outerwear (0.320)}	{Men’s Fashion (1.0)}
{3-D Graphics (0.678), Video Software (0.571), Graphics Software (0.570), Operating Systems (0.308), Business Accounting and Finance (0.351)}	{Computer Software and Applications (0.902), Business Accounting and Finance (0.351)}

trees in *.ete* format (see Figure 1) and obtain extended visualization as well (see Figure 3).

**Fig. 2.** An example of visualization of a cluster lifting over IAB taxonomy fragment.

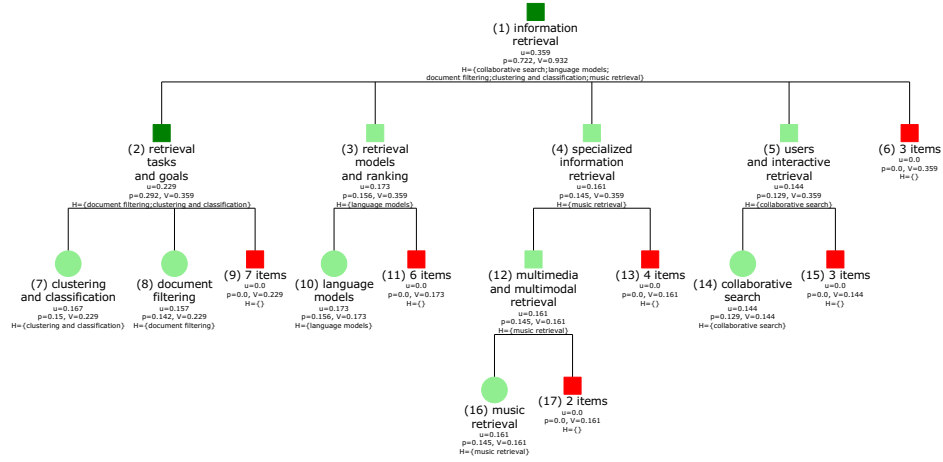


Fig. 3. An extended visualization of cluster lifting over TDS taxonomy. The legend is trimmed.

The comprehensive tutorial is located in GOT repository. The tutorial includes a number of examples for each stage of the hybrid methodology and files for experiments as well. A technical documentation for developers is available at ReadTheDocs⁸.

The recommended way to use the GOT is to work with it directly from the program code. However, the functionality of the taxonomies module can be used both from Python3 code and from the command line. This feature was implemented for convenience, based on practical experience with text collections and taxonomies of domains. To work with the GOT toolkit, one must have Python3 version 3.5 or newer.

For basic usage, one may skip the installation of the toolkit: it will be enough to clone the repository locally and run the programs from the repository folder. If one wants to fully import GOT modules into your Python code, is it possible to install the package via any Python3 package manager. The package has no special software requirements except *ete* package and *Qt* submodules for visualization. The detailed installation instructions are located in the GitHub repository.

4 Conclusion

We presented a novel Python3 software toolkit for taxonomy-based content analysis in collections of text documents. The toolkit follows a methodology, which provides for generalization, that is, mapping the meanings to more abstract levels. This feature is unique and cannot be simulated by using conventional approaches. Since the GOT toolkit provides for independent usage of its parts, the user gets flexible possibilities which can be used in other related areas.

⁸ <https://got-docs.readthedocs.io/>

Our future plans include extending the toolkit. Particularly, we are going to add implementations of other clustering methods. Also, we are going to work on implementing alternative approaches to generalization in taxonomies. Of course we are going to expand the usage of the toolkit in various applied areas.

References

1. Blei D., Ng A., Jordan M. Latent Dirichlet allocation. *Journal of Machine Learning Research*, 3: pp. 993-1022, January 2003.
2. Blei D. Probabilistic topic models. *Communications of the ACM*, **55** (4), pp. 77–84, 2012.
3. Bohr J., Dunlap R.E. Key Topics in environmental sociology, 1990-2014: results from a computational text analysis. *Environmental Sociology*. 3; **4** (2): pp. 181-95. 2018.
4. Chernyak E. An approach to the problem of annotation of research publications. *Proceedings of the eighth ACM international conference on web search and data mining*, ACM, pp. 429–434, 2015.
5. Chernyak E., Mirkin B. Refining a Taxonomy by Using Annotated Suffix Trees and Wikipedia Resources. *Annals of Data Science*, 2(1), pp. 61-82, 2015.
6. Dubov M. Text analysis with enhanced annotated suffix trees: Algorithms and implementation. In *International Conference on Analysis of Images, Social Networks and Texts*. Springer, Cham, 2015, pp. 308-319. 2015.
7. Frolov D., Nascimento S., Fenner T., Mirkin B. Parsimonious Generalization of Fuzzy Thematic Sets in Taxonomies Applied to the Analysis of Tendencies of Research in Data Science. *Information Sciences*. 2020. Vol. 512. P. 595-615. 2020.
8. Frolov D., Taran Z., and Mirkin B. A Method for Audience Extending in Programmatic Advertising by Using Parsimonious Generalization of User Segments. In *International Conference on Human Interaction and Emerging Technologies*, pp. 837-841. Springer, Cham, 2019.
9. Krippendorff K. Content analysis. In E. Barnouw, G. Gerbner, W. Schramm, T. L. Worth, & L. Gross (Eds.), *International encyclopedia of communication*, Vol. 1, pp. 403-407). New York, NY: Oxford University Press, 1989.
10. Luxburg van U. A tutorial on spectral clustering. *Statistics and Computing*, vol. 17, pp. 395-416, 2007.
11. Mirkin B., Nascimento S. Additive spectral method for fuzzy cluster analysis of similarity data including community structure and affinity matrices, *Information Sciences*, **183**, 1, pp. 16-34, 2012.
12. Mirkin B., Frolov D., Vlasov A., Nascimento S., Fenner T. A Hybrid Approach to the Analysis of a Collection of Research Papers. In *International Conference on Intelligent Data Engineering and Automated Learning 2020 Nov 4*. P. 423-433. Springer, Cham. 2020.
13. Pampapathi R., Mirkin B., Levene M. A suffix tree approach to anti-spam email filtering. *Machine Learning*, **65**(1), pp. 309–338, 2006.
14. Shi J., Malik J. Normalized cuts and image segmentation. *IEEE Transactions on pattern analysis and machine intelligence*, **22**(8), pp. 888-905. 2000.